GITHUB
GUIDE

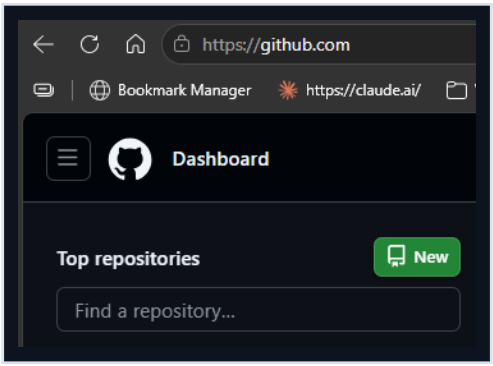
A simple guide to creating a repo, adding code, and committing changes — no dev background required.

DAN THE DIGITAL MAN

STEP 1

Create a Free GitHub Account

Go to **github.com** and sign up for a free account using your email address. This gets you a dashboard like the one below, where every repository you own or have access to shows up.

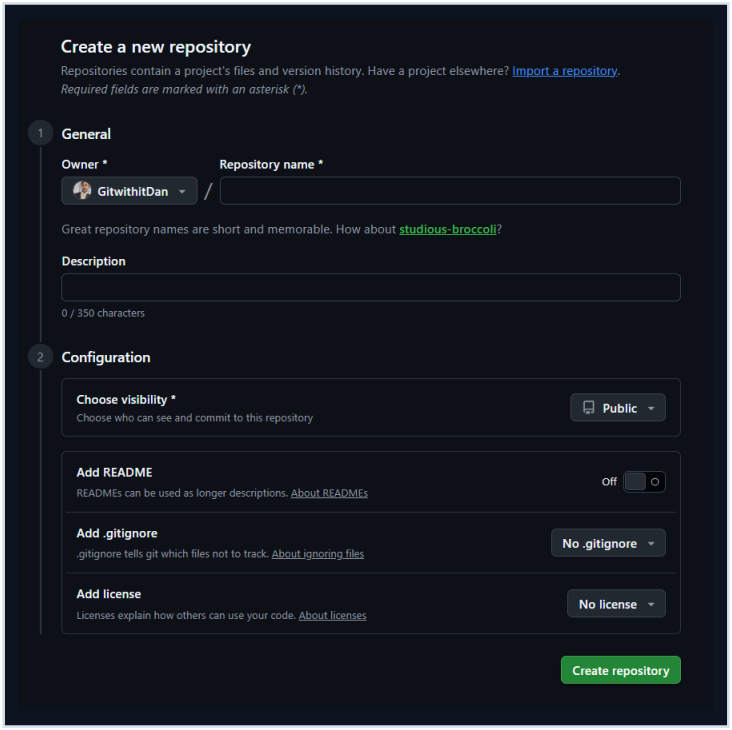


STEP 2

Create a New Repository

From your dashboard, click **New** to start a repository. A repository (or "repo") is just a project folder that stores your files and their full version history.

Give it a short, memorable name and click **Create repository**. You can leave README, .gitignore, and license off for a simple site repo.



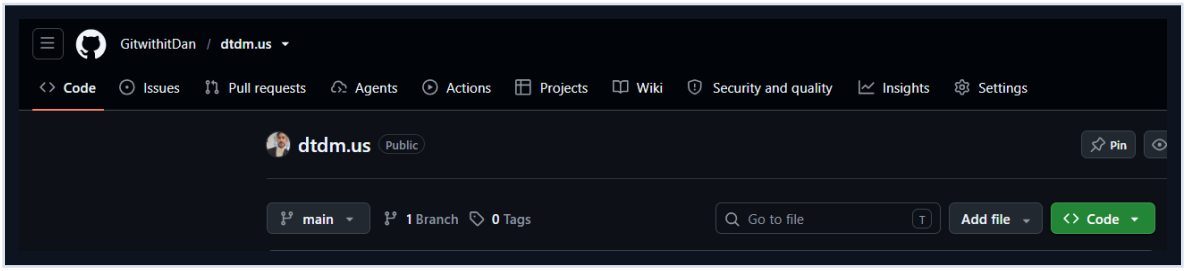
PUBLIC VS. PRIVATE

Public is fine for most projects — anyone can see the code, but only people you allow can edit it. **Private** hides the code entirely, but it can get in the way: some hosting platforms, integrations, and third-party connections either can't reach a private repo or need extra setup to do it. Unless you have a specific reason to keep the code hidden, start with Public and switch later if you need to.

STEP 3

Your Repo Home Page

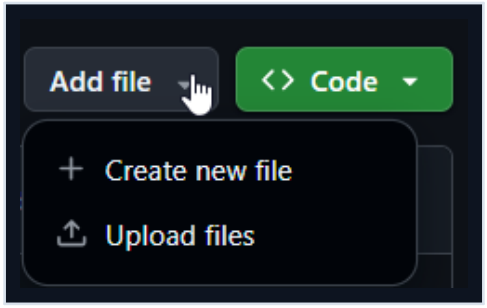
Once created, this is your repo's home screen. Across the top you've got Code, Issues, Pull requests, Actions, and Settings. For day-to-day site work, you'll live almost entirely in the **Code** tab.



STEP 4

Add or Upload Files

Click **Add file** to see your two options for getting code into the repo:



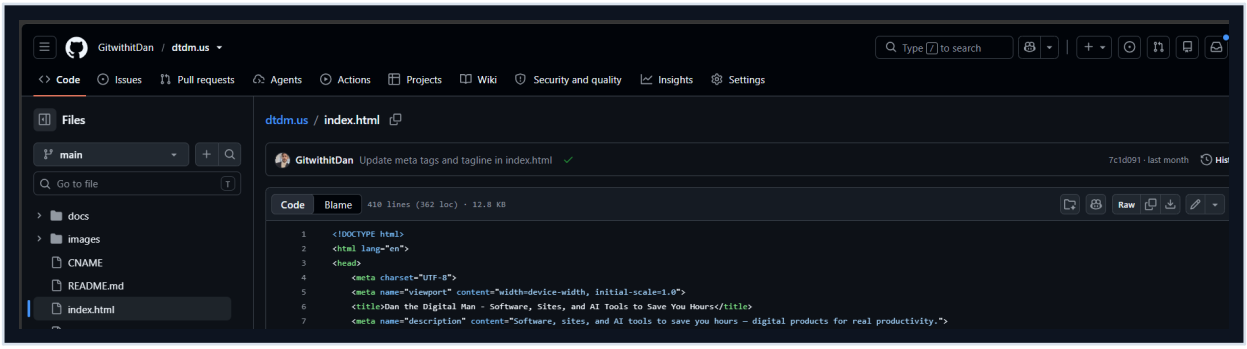
USE THE RIGHT OPTION

Create new file — use this when you're copy/pasting code directly (e.g. from a chat response) rather than working from a downloaded file.
Upload files — use this when you already have a file saved to your computer, such as one you downloaded from a chat or another tool.

STEP 5

Open a File to View or Edit It

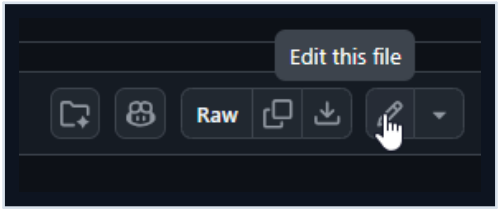
Click into any file in the file list — here, **index.html** — to see its contents, who last changed it, and its full history.



STEP 6

Find the Edit Button

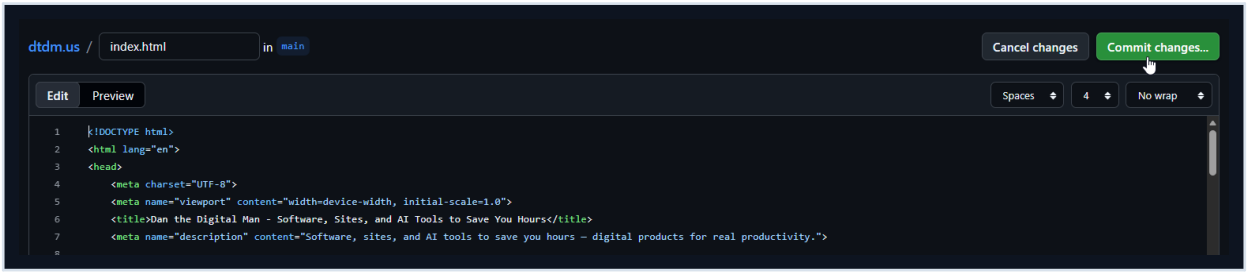
In the top-right corner of the file view, click the **pencil icon** to edit the file directly in the browser.



STEP 7

Edit the File

This opens the file in GitHub's built-in editor. Make your changes directly in the **Edit** tab, and use **Preview** to sanity-check formatted files like Markdown.



TIP: COPY/PASTE INSTEAD OF LINE-BY-LINE

After clicking edit, click anywhere in the code box to bring the cursor inside it. Then use **Ctrl+A** to select all text for easy deletion. Then paste in the updated code. This is often much easier than making line-by-line revisions — and it's worth telling your AI to output full updated files, not just code snippets or edits, so you always have a complete file to paste in.

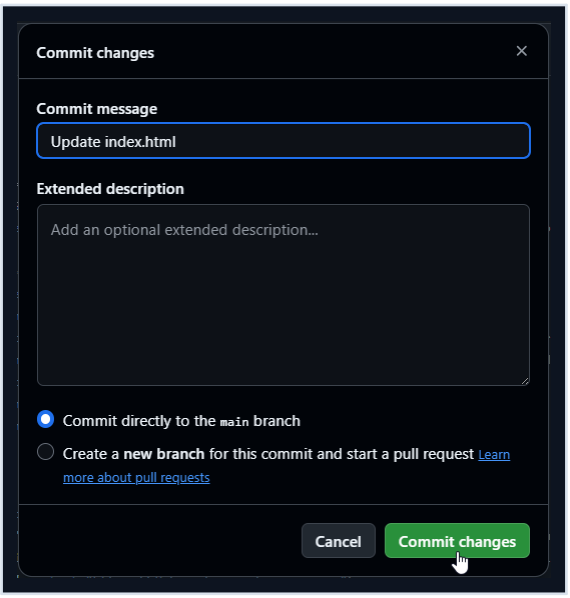
RENAMING & SUBFOLDERS

You can change the filename at the top of the editor. Type a **/** inside that filename to move the file into a subfolder as you save — for example, typing **docs/setup.html** creates (or uses) a **docs** folder and places the file inside it.

STEP 8

Commit Your Changes

When you're done editing, click **Commit changes...** in the top right. A dialog opens where you write a short commit message describing what changed, then confirm.



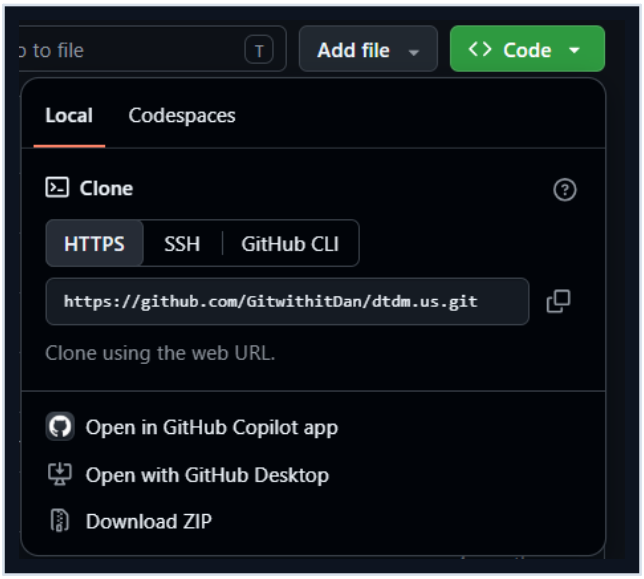
COMMITTING = SAVING

Choose **Commit directly to the main branch** for straightforward solo edits. This is the GitHub equivalent of hitting save — it locks in your change and adds it to the repo's history.

STEP 9

Download the Repo as a ZIP

Click the green **Code** button, then choose **Download ZIP** to grab a full copy of the current repo.



WHEN TO USE THIS

Download the ZIP any time you want to hand the current repo to an AI tool and ask it to make updates, or right after you've finished a round of edits and want a local backup of what's live.

PUTTING IT ALL TOGETHER

Example Project Workflow

Here's how these steps typically chain together on a real project, start to live site:

1. **Start the idea in Claude** — Describe what you want to build and ask Claude to output the code as complete, ready-to-use files — not snippets.
2. **Create the repo** — Follow Steps 1–4 above, then paste in or upload those files.
3. **Zip it and check your work** — Download the repo as a ZIP (Step 9) and upload it back into Claude to confirm every file is present and structured correctly.
4. **Keep developing** — As the idea grows, ask Claude for new or updated files, then edit and commit them into the repo using Steps 5–8.
5. **Move to hosting** — Once the project's ready to go live, it's time to put it on the web.

NEXT PHASE: HOSTING — TERMS TO LOOK OUT FOR

Not every project needs to go live. If you're building a personal tool just for yourself — a tracker, a dashboard, a one-off utility — you can often just open the HTML file straight from your computer, or run it locally, and skip hosting entirely. Hosting only matters once something needs to be reachable by other people, or by other tools/sites over the internet.

When a project does need to go live, here's the rough order of how much a project usually needs, and the terms to search for at each stage:

Quick look, no real hosting — Open the file in the repo, click the **Raw** button (same row as the pencil/edit icon from Step 6), and copy the URL from your browser's address bar. It's enough to open a basic HTML page in a browser to sanity-check it, but it's not a real address you'd hand someone or point a domain at.

GitHub Pages — free hosting built directly into GitHub. In the repo, click **Settings** (top nav), then **Pages** in the "Code and automation" section of the left sidebar. Under "Build and deployment," set Source to **Deploy from a branch**, then use the branch dropdown to select **main** (usually the only real option) and save. Once it deploys, the published URL appears at the top of that same Pages settings page — usually **username.github.io/repo-name**. Good fit for simple static sites — pages that don't need a database or server-side logic behind them.

Custom domain setup — once a project is hosted somewhere (GitHub Pages, Cloudflare, etc.), you can point a domain you own — like dtdm.us — at it, so people reach it by name instead of a platform-generated URL.

Cloudflare Pages / Workers — a step up from GitHub Pages. Pages handles static sites with more performance and configuration options; Workers adds the ability to run actual server-side code — useful once a project needs things like form handling, APIs, or logic that can't live in the browser alone.

Which of these applies depends entirely on the project — but these are the right terms to search for once you get there.

For more intensive projects — ones that need a real backend, persistent user data, or account logins — static hosting alone won't be enough. That's when services like **Railway** (app hosting), **MongoDB** (for expanded databases), or **Google Workspace** (for OAuth login capability) tend to come into play.